A Cloth Simulator That Knows When to Trust Itself: Detector-Gated Hybrid Neural–Physics Rollouts

Anonymous Author(s)
Affiliation
Address
email

Abstract

Cloth shows up everywhere in games and film, and lately in virtual try-on and robot manipulation too. The catch is speed. The high-fidelity solvers people trust, the Material Point Method (MPM) among them, inch forward in many tiny sub-steps to stay stable, which rules them out for anything interactive. Learned emulators flip that trade-off: they are fast, but they pile up errors over a long rollout, with nothing in the loop keeping the result physically honest. Classical solvers are accurate, but cost too much to lean on every frame. We take a middle path. A fast graph-network emulator does the ordinary work, and a cheap, interpretable deformation signal watches for the frames where it is likely to slip (the folds, the fast contacts) and routes just those to an implicit mass–spring fallback. What we did not expect is that the hand-off matters more than the detector. Switching between two solvers that are each perfectly stable on their own can land you worse off than either alone, and the fix is surprisingly simple: once the fallback engages, make it stay put for a while. On held-out cloth the hybrid runs 1.8–$2.0\times$ faster than full physics, trims the emulator's positional drift by 30.8%, and the detector flags hard frames at 0.98 AUROC. We also report the tidy fixes that looked obvious and did not work.

1 Introduction

Picture a character's skirt drifted by the wind, or a flag snapping on a pole. Rendering such *cloth* on screen almost always means simulating the physics: springs and masses in the classic setup [1], or a continuum discretized onto a background grid, which is what the Material Point Method does [10, 8, 17]. Cloth then drove the method in its own directions. Thin-shell formulations, anisotropy along warp and weft, contact and self-contact resolved right on the particle–grid transfer [7, 20], all of it layered on top of the collision machinery cloth has always needed [5]. While it works, it is not cheap. Explicit schemes require tiny time steps or they blow up. Implicit ones trade that for large linear solves. Either way, a single frame is many sub-steps, and the cost climbs quickly as you refine the mesh to catch fine wrinkles [22]. For an offline render, this works just fine. For something you want to interact with and watch respond, it falters. That is what sends people searching for a cheaper stand-in that still moves like fabric.

The obvious stand-in is a network that has watched enough simulation to imitate it. Graph networks are the default choice. They learn particle dynamics by message passing [15], and MeshGraphNets does the mesh-based version, which already covers cloth [13]; both rest on the broader graph-network framework [2]. Related particle methods learn deformable and fluid dynamics directly [11, 19]. Cloth has a specialized branch of its own: the networks that predict garment drape and dynamics from body pose, sometimes with physics folded into the architecture or the loss [23, 3, 16, 6]. A different approach keeps the numerical solver and learns only a correction

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

to it [18], which differentiable simulators make practical by exposing gradients straight through the solver [9]. These are all genuinely fast. The problem surfaces over time: a pure-neural rollout feeds its own output back in, so a small slip this step becomes the input to the next, and the trajectory drifts, with nothing holding it to the physics [15]. The natural remedy is running the emulator most of the time and dropping back to a real solver on the hard frames, and we are not the first to reach for it; the closest precedent runs a neural MPM for fluids with a classical solver as a safety net [21], and that is precisely the design we carry over to cloth. Where we change the approach is crucial. That work, and hybrids like it, mostly ask how accurately you can *spot* the hard frames; they don't consider what happens at the seam, when one solver hands control to the other. The seam, it turns out, is the highlight of the story.

Three pieces make up the system. An MLS-MPM cloth solver [8, 10] is the reference, generating the training data and defining what "correct" even means here. The emulator is a MeshGraphNets-style graph network [13] trained on short rollouts of its own predictions. It uses the pushforward trick from the PDE-solver literature [4] and a method closely related to the training noise used in GNS [15]. The fallback mechanism is a plain implicit mass–spring step [1], inspired by the seed hybrid's classical safeguard [21]. A lightweight detector, which monitors the frame-to-frame change in a local strain estimate, determines when this fallback is triggered. One point is worth stating outright, as it colors how the entire system should be evaluated: the fallback is not the ground truth. It is a low-fidelity model in its own right, meaning the hybrid approach stitches together two different approximations, neither of which is the reference. How long the fallback remains active and how often control changes hands, is where quality is won or lost, far more than in the detector that trips it. We also tried the standard fixes you would typically reach for first: fine-tuning the emulator on the fallback's states in a DAgger-style loop [14], and blending the two solvers across the switch rather than applying a hard cut. Neither approach helped, and explaining why they did not succeed is part of what we are reporting. For speed, inference runs in fp16 [12], which we find essentially lossless in this case.

Contributions. There are three concrete contributions:

- **A fast, faithful hybrid cloth simulator.** We introduce a graph-network emulator backed by a low-fidelity implicit fallback that activates only when triggered by an interpretable deformation signal. This approach is 1.8–$2.0\times$ faster than the reference MPM, shows 30.8% less positional drift on a held-out drape, and achieves a detector score of 0.98 AUROC.
- **The hand-off, not the detector, governs the rollout.** Naive switching can perform worse than using either solver alone; we found that a minimum-dwell (hysteresis) rule provides the most dependable solution. Alternative approaches, such as DAgger-style fine-tuning and cross-solver blending, fail because the emulator and the fallback occupy different dynamics manifolds.
- **An honest account.** We provide a working GPU implementation with the speed-accuracy trade-off measured end-to-end, alongside a candid record of the approaches that did not work rather than setting them aside.

2 Method

Reference and data. The reference is a Taichi MLS-MPM thin-shell cloth solver [8, 9] configured on a 64×64 sheet ($4{,}096$ particles). We utilized two scenarios: *drape* (where the sheet falls and settles without pins) and *collision* (where the sheet is pinned at two corners and drapes over a moving sphere). Wind effects were deferred, as they were not stable enough at full resolution to be trusted for training data. We trained the model on a balanced dataset of $3{,}000$ clips ($1{,}500$ per scenario), with each clip storing per-frame positions, velocities, and accelerations.

Neural emulator. We use a MeshGraphNets-style graph network [13]: $L{=}5$ message-passing steps, hidden width 128, about 847k parameters. The nodes carry the particle state and the edges follow the mesh. The network maps (x, v) to per-particle acceleration, which is then advanced using a semi-implicit Euler step. We removed the deformation gradient F from the input because it is codimensional for a thin shell and its determinant drifts, which negatively impacted performance. Training uses the pushforward curriculum [4], which unrolls K steps on the model's own predictions

and backpropagates through the integration, ramping $K \to 5$. On a practical note, the naive unroll holds the autograd graph for the whole batch and runs out of memory by $K{=}2$. Taking the backward pass per clip reduces peak memory from $B{\times}K$ to $1{\times}K$ and makes the curriculum feasible.

Detector and routing. We want a per-frame signal for *where the emulator will drift*, using only what a deployed no-F rollout actually has: positions. Ours is a position-based strain proxy, a local deformation gradient fit by least squares from a particle's neighbourhood, tracked by its frame-to-frame rate of change. It is deliberately *not* the cosine-of-acceleration signal prior hybrids default to, which targets locally hard dynamics like contact that our emulator already handles. The rollout runs the emulator by default and, when the proxy crosses a threshold, substitutes a fallback step. The subtlety is when to hand back: a per-frame switch churns control, and that churn is what hurts (Sec. 3), so we add *hysteresis*, staying in fallback until the signal drops well below the trigger. This keeps control in one solver for a contiguous stretch; a minimum-burst variant (stay $\geq K$ frames) behaves identically and makes the mechanism plain.

Physics fallback. An implicit mass–spring Euler step on the same particles treated as mesh nodes [1]. Our initial CPU solver (using modified preconditioned conjugate gradient) was correct but slow (119 ms/frame). To improve performance, we developed a matrix-free GPU rewrite that forms no explicit system matrix and runs a fixed-iteration conjugate gradient solver under `torch.compile`. This optimization reduced the compute time to 3.55 ms, which is faster than the MPM reference.

3 Experiments

Protocol. Accuracy is measured using fp32 on held-out clips. Speed is evaluated on an A10G GPU with warmup and device synchronization. For the hybrid model, we tune the knobs (threshold, hysteresis exit ratio) on one split and report results on a *disjoint* split to ensure the metrics are independent of the tuning set. Positional error is calculated as per-particle L2 against the MLS-MPM reference. The detector is scored by AUROC against $y_t = \mathbf{1}[\,\mathrm{L2}(t) > 0.10\,\mathrm{m}\,]$.

Surrogate accuracy. Table 1 shows the emulator's performance. The results split by scenario, contrary to our initial expectations. On *collision* scenario, the case we worried about most, the emulator tracks tightly and nearly conserves energy (drift 0.017). On the *drape* scenario, it stays stable but the swing slides out of phase late in a long rollout. As a result, the final L2 and energy numbers reflect phase mismatch rather than a blow-up. This asymmetry is exactly what the detector and fallback mechanisms are designed to cover.

Detector. The position-only strain proxy separates drift frames at 0.980 (for drape) and 0.985 AUROC (for collision). This matches the F-based signal but is computable from predicted positions, meaning it survives deployment (Fig. 2). Conversely, the cosine-of-acceleration signal that prior hybrids use scores 0.65 on drape, which is the wrong target. One trap worth flagging: a classifier on all features *pooled* across scenarios scores well by quietly guessing the scenario while performing worse within each. Therefore we report per-scenario, where "strain proxy plus threshold" serves as the detector.

Speed. Table 3 has the performance numbers from our latest benchmarks. Initially, the fp32 net (9.9 ms) is actually *slower* than well-tuned Taichi MPM (5.6 ms), and the CPU fallback was significantly slower at (119 ms). However, this was an implementation issue rather than a conceptual one. By using fp16 with `torch.compile` the network execution time dropped to 2.1 ms (lossless per step, 0.07% acceleration error), and the matrix-free GPU fallback to 3.55 ms. End-to-end in the rollout loop, the hybrid approach is 1.8–$2.0\times$ faster than the full MPM. While this is lower than our initial target of 3–$5\times$ it represents a real, end-to-end measured speedup.

Hybrid accuracy, and the hand-off. Here is the interesting finding regarding the solvers. Each solver alone performs well on drape (pure neural ends at L2 0.190, pure fallback at 0.124). However, switching every frame the raw detector fires results in an L2 of 0.389, worse than *both* individual approaches. The solvers are individually stable; the switching is what causes the performance to drop. Each switch hands one solver a state produced by the other, which is off its own manifold, causing the mismatch to compound. Figure 1c measures this effect: immediately after a switch the

Table 1: Neural emulator alone, held-out (400-step rollouts). Collision is the strong case; drape drifts out of phase late.

metric	drape	collision
one-step accel. err.	0.405 m/s^2	
L2 @ 200 (m)	0.017	0.005
L2 final (m)	0.19	0.077
energy drift	0.70	**0.017**

Table 2: Held-out drape, hysteresis tuned on 17 clips, reported on 18 disjoint clips. Hybrid matches pure-fallback accuracy on about half the physics.

policy	L2 final (m)	vs. neural
neural	0.186	n/a
fallback-only	0.128	$+31.3\%$
hybrid (hyst.)	**0.129**	$\mathbf{+30.8\%}$

Table 3: Per-frame wall-clock, 4,096 particles, A10G. Fair basis: neural is one forward step ($\mathrm{dt}{=}10^{-3}$); MPM is 10 sub-steps ($\mathrm{dt}{=}10^{-4}$) for the same physical time; fallback is one stable step.

	full-MPM	neural fp32	neural fp16+compile	fallback CPU	fallback GPU
per-frame	5.6 ms	9.9 ms	2.1 ms	119 ms	3.55 ms
vs. full-MPM	$1.0\times$	$0.6\times$	$2.65\times$	n/a	n/a

Hybrid, end-to-end in the rollout loop: $\mathbf{1.8}$–$\mathbf{2.0}\times$ faster than full-MPM.

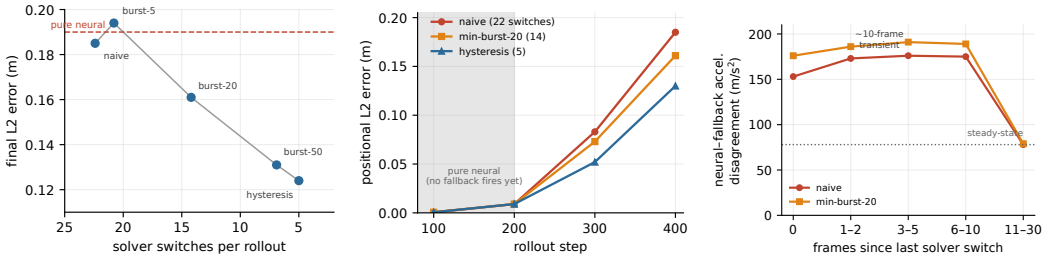
two solvers' accelerations, evaluated on the same state, disagree 1.5–$2.7\times$ more than in steady state, relaxing over about 10 frames. Every hand-off injects a $\sim$10-frame transient; constant switching means paying this penalty continuously. This insight reframes our approach. At a *matched* fallback budget, error falls monotonically as switches drop (Fig. 1a). Naive routing requires $\sim$22 switches to reach an L2 of 0.185, while sticky hysteresis needs $\sim$5 switches to reach 0.124, a third lower on the *same* physics. Figure 1b shows the same policies over time, which remain identical until the fallback first fires (around step 200), then fanning out by how much they churn. When the hysteresis is tuned on a disjoint split, the hybrid model holds up well on unseen clips (Table 2): It achieves $+30.8\%$ over pure neural approach at a 49% fallback rate, statistically matching accuracy of pure-fallback approach at half of its physics cost. On collision, the detector fires on 0% of frames, correctly, since the emulator there (0.033) already outperforms the fallback (0.28), so the hybrid quietly reduces to the emulator.

What did not work. Two natural attempts to *remove* the per-hand-off cost, ultimately failed for the same reason. They are worth noting because they are the first solutions a reader will consider. *DAgger fine-tuning* [14]: Training the emulator on states visited by the fallback mechanism fails to resolve the issue. Querying the MPM reference at these states is unstable because they lie off the MPM manifold, resulting in one-step accelerations of 250–1600 m/s^2 compared to a physical limit of 10–30 m/s^2. Alternatively, using the fallback's *own* accelerations introduces a distributional incompatibility with the MPM data, causing the normalized loss to explode. *Blending*: Crossfading the two next-states rather than executing a hard cut yields *worse* results at a matched budget. A convex blend of states from different manifolds lands off both, which prolongs off-manifold exposure. Ultimately, both failures demonstrate a structural gap between two distinct dynamics. The solution is to cross this gap less frequently rather than attempting to smooth the crossing.

4 Limitations and Conclusion

Here are a few caveats regarding the current results: The speedup is $\sim 2\times$, rather than the 3–5$\times$ we first targeted. Results cover drape and collision at a 64×64 resolution. Wind has been deferred, and larger resolutions have been timed but not accuracy-validated. The accuracy gain *needs* hysteresis, so it is a property of the schedule rather than an inherent benefit. "Matches physics" indicates alignment with the low-fidelity mass–spring *fallback*, and not the exact MPM reference. A single scenario-agnostic detector remains future work.

Those aside, the artifact stands: a graph-network cloth emulator with an interpretable, position-only drift detector and a cheap implicit fallback that runs 1.8–$2.0\times$ faster than full MPM while cutting drift by 30.8% on held-out cloth. The transferable lesson is the one we did not anticipate. When you couple a neural emulator to a physics solver, the hand-off between them is a first-class design



(a) (b) (c)

Figure 1: The hand-off, not the detector, governs the rollout. **(a)** At a matched fallback budget, final error falls monotonically as switches drop; sticky routing beats fragmented routing on the *same* physics. **(b)** The policies are identical until the fallback first fires (∼step 200), then separate by how much they churn. **(c)** The mechanism: after each switch the two solvers disagree well above their steady-state level for ∼10 frames, a transient that fragmented switching re-triggers constantly.

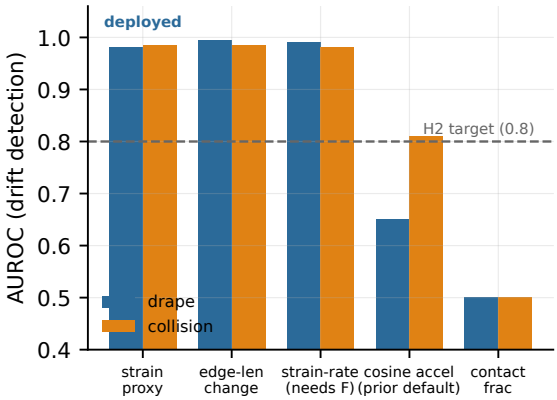


Figure 2: Detector separability (AUROC) by feature and scenario. The deployable position-based strain proxy matches the F-based signal and clears the H2 target; the cosine-of-acceleration signal used by prior hybrids does not.

variable. Switching too often can cause two good solvers to make a bad rollout. The reliable fix is not a cleverer blend or more fine-tuning (both of which failed) but simply switching less. For anyone building a simulator meant to know when to trust itself, that is the part worth remembering.

References

[1] David Baraff and Andrew Witkin. Large steps in cloth simulation. In *SIGGRAPH*, pages 43–54, 1998. doi: 10.1145/280814.280821.

[2] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.

[3] Hugo Bertiche, Meysam Madadi, and Sergio Escalera. PBNS: Physically based neural simulation for unsupervised garment pose space deformation. *ACM Transactions on Graphics (SIGGRAPH Asia)*, 40(6):1–14, 2021. doi: 10.1145/3478513.3480479.

[4] Johannes Brandstetter, Daniel E. Worrall, and Max Welling. Message passing neural pde solvers. In *International Conference on Learning Representations (ICLR)*, 2022.

[5] Robert Bridson, Ronald Fedkiw, and John Anderson. Robust treatment of collisions, contact and friction for cloth animation. *ACM Transactions on Graphics (SIGGRAPH)*, 21(3):594–603, 2002. doi: 10.1145/566570.566623.

[6] Artur Grigorev, Bernhard Thomaszewski, Michael J. Black, and Otmar Hilliges. HOOD: Hierarchical graphs for generalized modelling of clothing dynamics. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16965–16974, 2023.

[7] Qi Guo, Xuchen Han, Chuyuan Fu, Theodore Gast, Rasmus Tamstorf, and Joseph Teran. A material point method for thin shells with frictional contact. *ACM Transactions on Graphics (SIGGRAPH)*, 37(4):1–15, 2018. doi: 10.1145/3197517.3201346.

[8] Yuanming Hu, Yu Fang, Ziheng Ge, Ziyin Qu, Yixin Zhu, Andre Pradhana, and Chenfanfu Jiang. A moving least squares material point method with displacement discontinuity and two-way rigid body coupling. *ACM Transactions on Graphics (SIGGRAPH)*, 37(4):1–14, 2018. doi: 10.1145/3197517.3201293.

[9] Yuanming Hu, Luke Anderson, Tzu-Mao Li, Qi Sun, Nathan Carr, Jonathan Ragan-Kelley, and Frédo Durand. Difftaichi: Differentiable programming for physical simulation. In *International Conference on Learning Representations (ICLR)*, 2020.

[10] Chenfanfu Jiang, Craig Schroeder, Joseph Teran, Alexey Stomakhin, and Andrew Selle. The material point method for simulating continuum materials. In *ACM SIGGRAPH 2016 Courses*, pages 1–52, 2016. doi: 10.1145/2897826.2927348.

[11] Yunzhu Li, Jiajun Wu, Russ Tedrake, Joshua B. Tenenbaum, and Antonio Torralba. Learning particle dynamics for manipulating rigid bodies, deformable objects, and fluids. In *International Conference on Learning Representations (ICLR)*, 2019.

[12] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.

[13] Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W. Battaglia. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations (ICLR)*, 2021.

[14] Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15, pages 627–635, 2011.

[15] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. Learning to simulate complex physics with graph networks. In *International Conference on Machine Learning (ICML)*, pages 8459–8468, 2020.

[16] Igor Santesteban, Miguel A. Otaduy, and Dan Casas. SNUG: Self-supervised neural dynamic garments. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8140–8150, 2022.

[17] Alexey Stomakhin, Craig Schroeder, Lawrence Chai, Joseph Teran, and Andrew Selle. A material point method for snow simulation. *ACM Transactions on Graphics (SIGGRAPH)*, 32 (4):1–10, 2013. doi: 10.1145/2461912.2461948.

[18] Kiwon Um, Robert Brand, Yun (Raymond) Fei, Philipp Holl, and Nils Thuerey. Solver-in-the-loop: Learning from differentiable physics to interact with iterative pde-solvers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[19] Benjamin Ummenhofer, Lukas Prantl, Nils Thuerey, and Vladlen Koltun. Lagrangian fluid simulation with continuous convolutions. In *International Conference on Learning Representations (ICLR)*, 2020.

[20] Joshuah Wolper, Yunuo Chen, Minchen Li, Yu Fang, Ziyin Qu, Jiecong Lu, Meggie Cheng, and Chenfanfu Jiang. Anisompm: Animating anisotropic damage mechanics. *ACM Transactions on Graphics (SIGGRAPH)*, 39(4):1–16, 2020. doi: 10.1145/3386569.3392428.

[21] Jingxuan Xu, Hong Huang, Chuhang Zou, Manolis Savva, Yunchao Wei, and Wuyang Chen. Hybrid neural-mpm for interactive fluid simulations in real-time. *arXiv preprint arXiv:2505.18926*, 2025.

[22] Diyang Zhang, Zhendong Wang, Zegao Liu, Xinming Pei, Weiwei Xu, and Huamin Wang. Physics-inspired estimation of optimal cloth mesh resolution. In *ACM SIGGRAPH 2025 Conference Papers*, 2025. doi: 10.1145/3721238.3730619.

[23] Zhiwei Zhao. A physics-embedded deep learning framework for cloth simulation. *arXiv preprint*, 2024. Preprint, 2024; venue/arXiv id to be verified.